WAC: collecting Windows without leaving a trace

Aerospace Cyber Defence Centre of Excellence (CEC), French Air and Space Force Academy

A collection is made on a machine that is also evidence. The tool that takes it must take everything, write nothing there, and be able to prove it. WAC (Windows Artefacts Collector), free software written in C++, collects 69 artefacts of a Windows workstation by raw reading, converts them to JSON on the analysis workstation, and seals and logs every one of its operations. Its results were confronted, value by value, with those of KAPE and of Eric Zimmerman's tools.

What is collected

A single standalone executable, run from a USB key, reads the NTFS volumes raw: the `$MFT` and the directory indexes, without opening a file. Locked files (hives, logs) are read like the others, and no access date changes. 69 artefacts: system and per-profile hives (the whole SAM, BAM, UserAssist, ShimCache, Amcache, shellbags, USB), `$MFT`, `$UsnJrnl`, `$LogFile`, event logs with the resources that give their text, Prefetch, shortcuts, jump lists, SRUM, recycle bin, scheduled tasks, WMI, persistence, Defender, browsers (history, downloads, sessions and closed tabs), mail, DPAPI keys, the FAT and exFAT volumes of USB keys. The machine's executables are read and their Authenticode signature checked in memory: an authentic Microsoft binary is recorded by its fingerprints only, any other is copied. Against KAPE (`KapeTriage`), WAC takes everything KAPE takes — 5 `desktop.ini` aside — and 790 more files, in 41 s against 44.

A procedure kept

WAC *collects* on the examined machine or *converts* on the analysis workstation, never both. Every exhibit enters a sealed **exhibit store**: its manifest gives, for each, MD5, SHA-1 and SHA-256, the original's eight NTFS dates (`$STANDARD_INFORMATION` and `$FILE_NAME`), its MFT entry and the reading method; the manifest's fingerprint seals it. After the initial copy, the store is neither read nor written again: the conversion first checks the seal and every fingerprint, refuses a retouched exhibit, then works on a working copy, where the logs of the hives and ESE databases are replayed. Every operation is written to the **investigation log** (`investigation.json`): instant, target, result, the trace it is expected to leave, and the time of each step. Two shipped scripts recheck a store without WAC.

JSON, not CSV

One file per artefact, typed values (numbers, booleans, nested lists), every date to 100 ns in UTC and in the suspect's local time, with the offset in force *at that date*. A field without a value is not emitted: no zero, no 1601 date that would read as data. Each event's text is rendered as Windows displays it. The result loads as is into an analysis tool.

No trace, measured

On a frozen clone of the test machine, each tool is run from a key, under Sysmon, the NTFS change journal bounded before and after. WAC writes **no file** on C:, **no registry value**, no event in Windows' logs; it starts no process and sends nothing on the network. What remains are the Amcache entry and the ShimCache entry Windows creates for any program, and what a monitoring records of its execution (Sysmon: the process, two raw reads of a volume). Under the same conditions, KAPE writes to the user's hive, and its modules 28 files on C:. The build script refuses a `WAC.exe` that would import a function able to start a process.

Tests that can fail

An autonomous cycle, in a Windows 11 virtual machine, lays witnesses of known value (files, registry, browsers, mail, a 261-character path), collects, converts, and passes 494 checks. Each confronts WAC with an independent source: Windows itself (`Get-WinEvent`, `Get-FileHash`, registry), `dissect.esedb`, Python's SQLite and `lz4`. A second conversion must give the same JSON files, byte for byte. Against Eric Zimmerman's eleven tools, on the same exhibits, each difference was settled by such a source: WAC's values are the same, or more accurate (RECmd rolls hives back to an older state, AmcacheParser invents 3,450 values, EvtxECmd is wrong on ten). The confrontation also revealed 28 defects of WAC, all fixed with the check that would have caught them.

In figures

Artefacts collected	69
Collection without the executables (KAPE: 44 s)	41 s
Whole conversion, store checked	80 s
Files written on C: by WAC	0
Checks of the test cycle	494
Reference tools confronted	12
WAC defects found and fixed	28

Availability

Version 2.0.0, Windows 10 and 11 (x64), GPL 3.0 licence: `https://github.com/dev-cec/WAC`. Manuals, validation, harmlessness and comparison files in English and French.